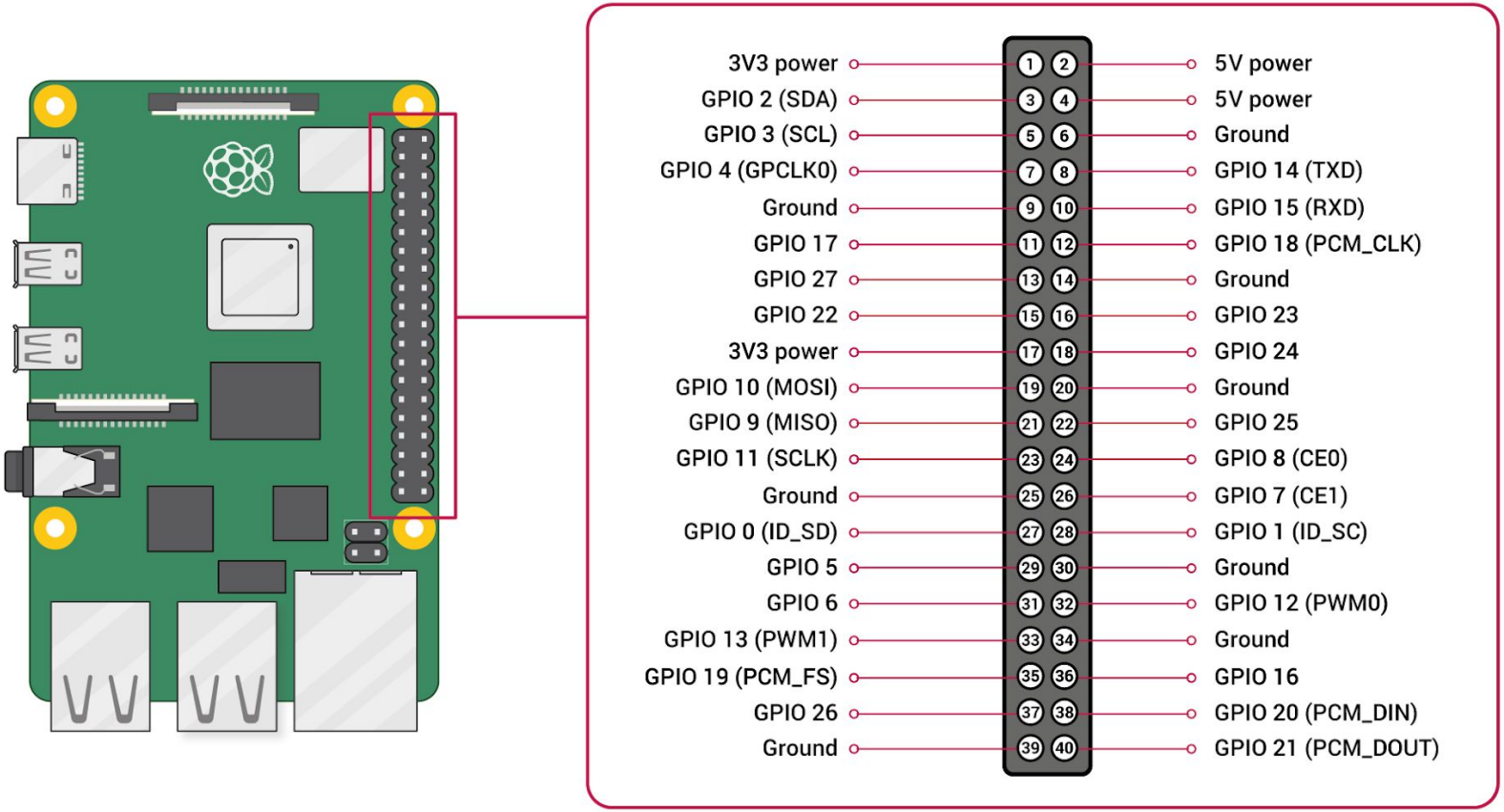
Sistemas Operacionais Embarcados

GPIO Polling

Pinos GPIO



- Pinos GPIO trabalham em 3,3V
- RPis 1A e 1B têm somente os 26 pinos superiores da lista
- Legendas em parênteses indicam funções alternativas para o mesmo pino

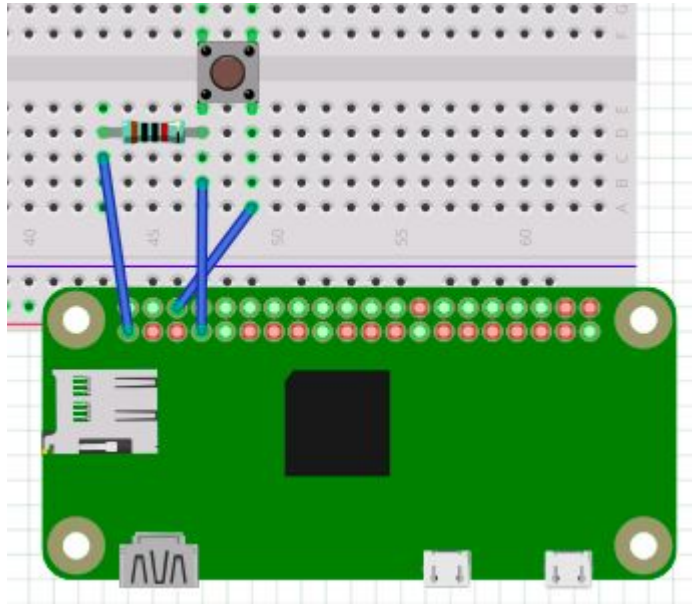
Detecção de eventos em GPIO (libgpiod)

- Em vez de leitura contínua (*polling* ativo), podemos usar eventos

- A biblioteca libgpiod permite:
  - Detectar bordas de subida (*rising edge*)
  - Detectar bordas de descida (*falling edge*)
  - Bloquear a execução até ocorrer um evento

- Mais eficiente que *polling* contínuo:
  - Menor uso de CPU
  - Melhor tempo de resposta

- Baseado em dispositivos:
  - `/dev/gpiochipN`



Detecção de eventos em GPIO (libgpiod)

- Eventos com libgpiod (C)

```
struct gpiod_line_event {
   struct timespec ts;   // timestamp do evento
   int event_type;       // tipo do evento
};
```

- Tipos de evento:
  - `GPIOD_LINE_EVENT_RISING_EDGE`
  - `GPIOD_LINE_EVENT_FALLING_EDGE`

Espera por eventos

- Podemos bloquear o programa até ocorrer um evento:

```
gpiod_line_event_wait(line, timeout);
```

- O programa fica em espera até:
  - ocorrer uma borda ou
  - atingir o *timeout*

- Evita leitura contínua (*busy waiting*)

Configuração de eventos

- Antes de esperar eventos, devemos configurar a linha:

```
gpiod_line_request_both_edges_events(line, "app");
```

- Outras opções:
  - `gpiod_line_request_rising_edge_events()`
  - `gpiod_line_request_falling_edge_events()`
- A linha passa a gerar eventos automaticamente

Leitura de eventos

- Após detectar um evento:

```
gpiod_line_event_read(line, &event);
```

- O evento contém:
  - tipo (subida ou descida)
  - *timestamp*

- Permite processamento preciso de sinais digitais

Identificação do evento

- `event.event_type` indica:
  - `GPIOD_LINE_EVENT_RISING_EDGE`
  - `GPIOD_LINE_EVENT_FALLING_EDGE`
- Exemplo:

```
if (event.event_type == GPIOD_LINE_EVENT_RISING_EDGE)
{
    // botão pressionado
}
```

Função de espera

```
int gpiod_line_event_wait(struct gpiod_line *line,
    const struct timespec *timeout);
```

- Parâmetros:
  - `line`: linha GPIO monitorada
  - `timeout`: tempo máximo de espera

- Retorno:
  - 0 → *timeout*
  - 1 → evento ocorreu
  - -1 → erro

Configuração de bordas

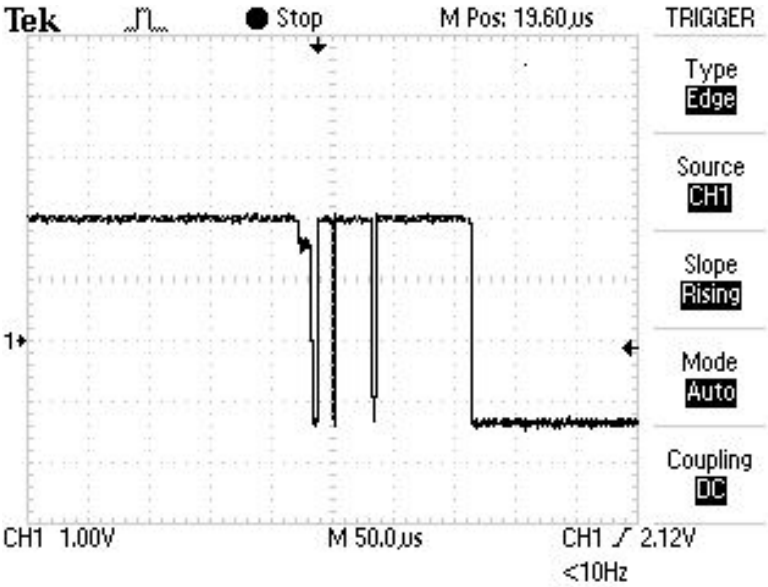
- A detecção de bordas é feita na requisição da linha:

  `gpiod_line_request_both_edges_events()`

- Não é necessário configurar arquivos como no sysfs

- A API já entrega eventos diretamente ao usuário

Debouncing



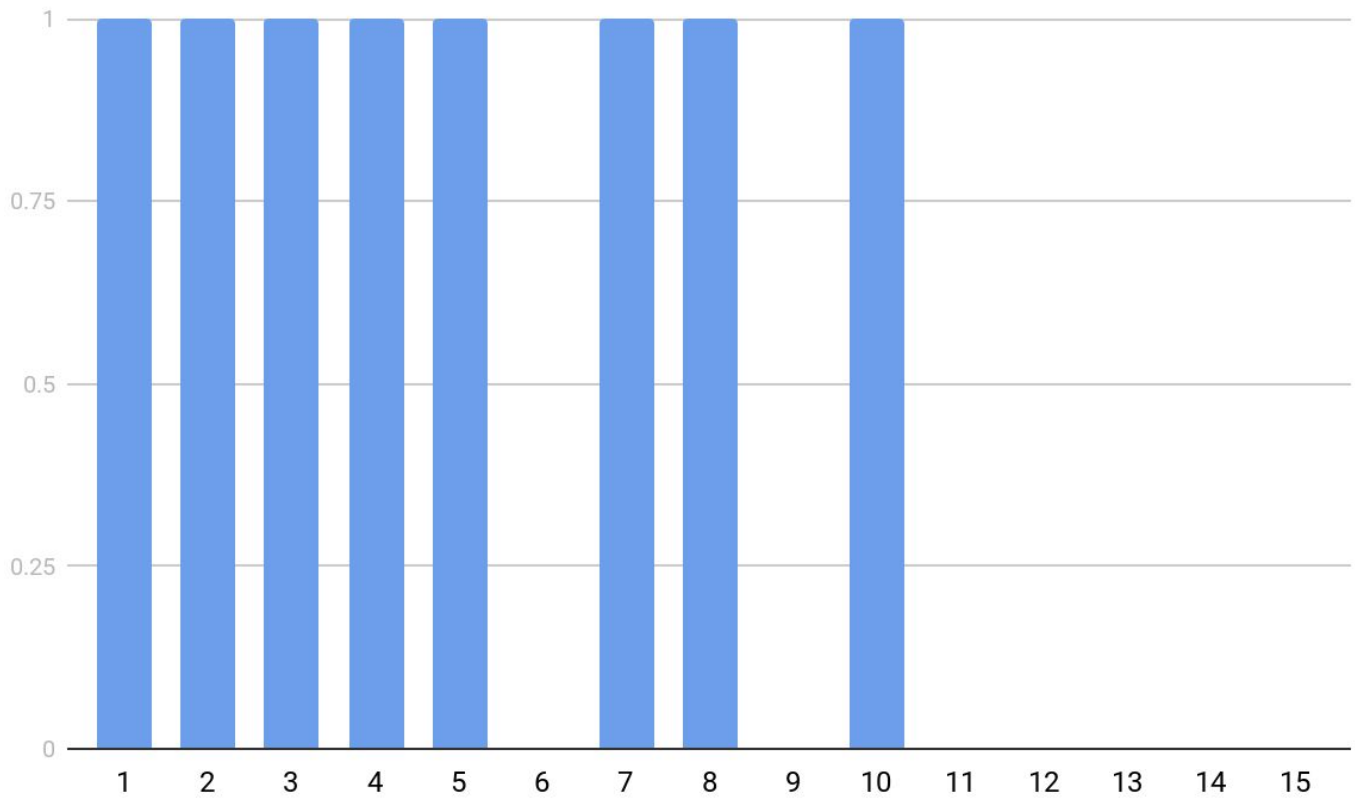
Botões e chaves mecânicas tipicamente sofrem com o problema de *bouncing*, causando leituras espúrias

Dentre as diversas formas de resolver este problema, podemos:

- Acrescentar um atraso fixo após a detecção de uma borda
- Fazer um filtro média-móvel

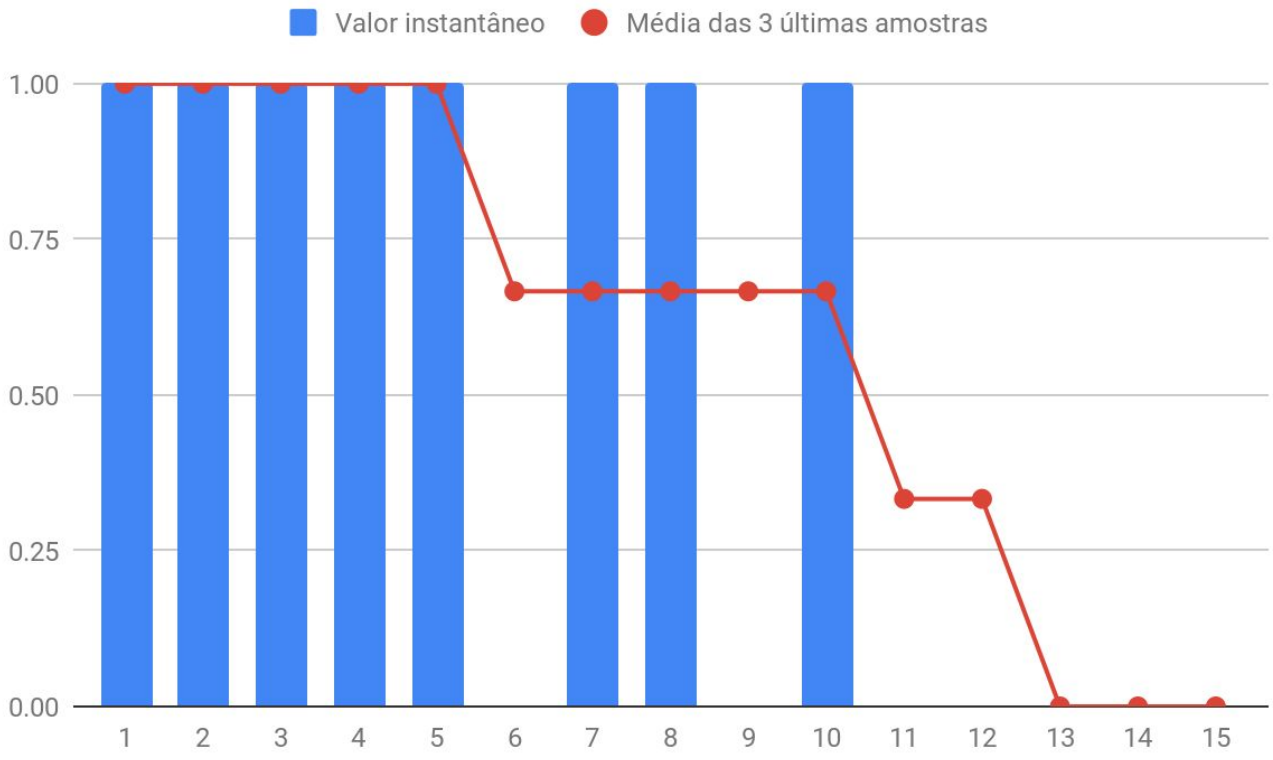
Filtro média móvel

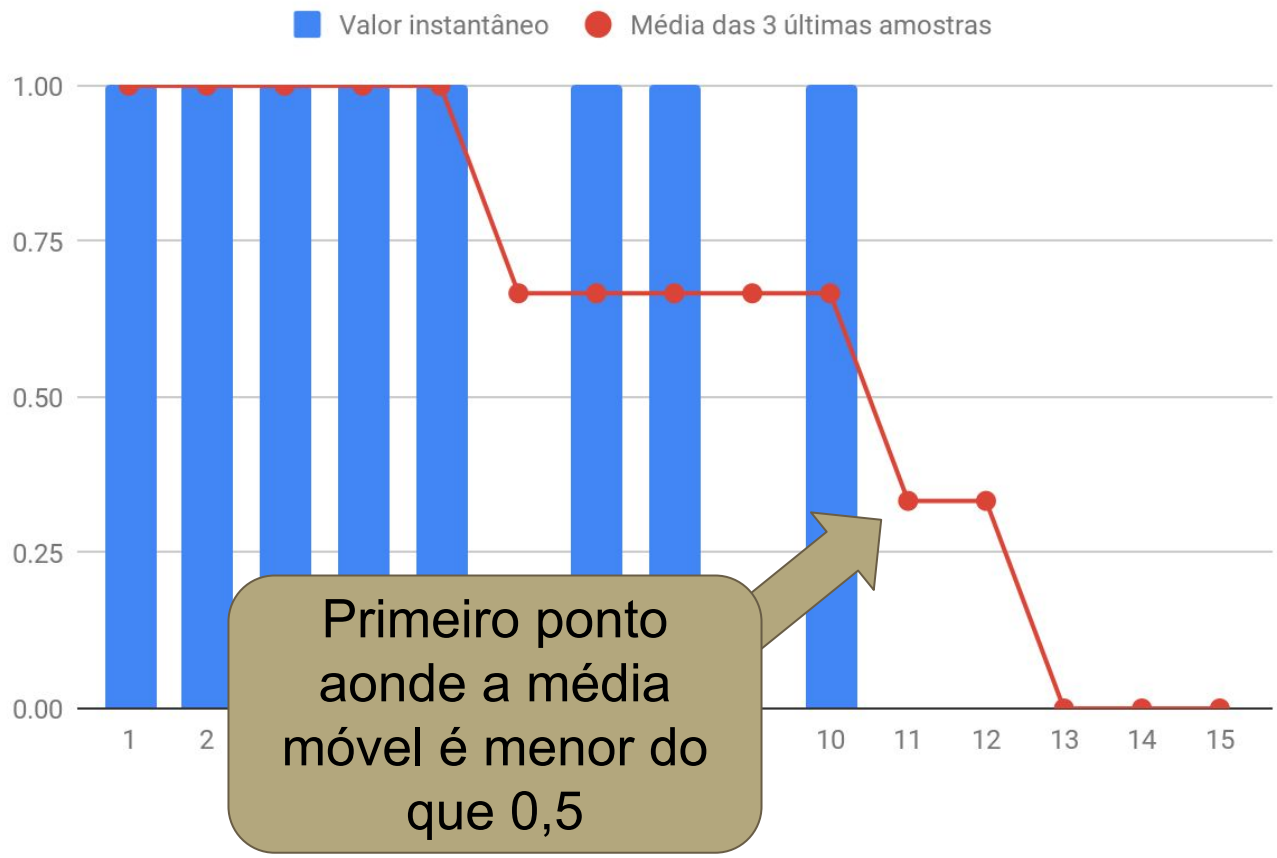
Considere o sinal digital a seguir:



Filtro média móvel

Ao invés de usarmos os valores instantâneos, podemos calcular a média das últimas N amostras para decidirmos se o botão foi pressionado



Filtro média móvel

Ao invés de usarmos os valores instantâneos, podemos calcular a média das últimas N amostras para decidirmos se o botão foi pressionado